

Xam

Diseño e Implementación de un DSL
para Generación de Exámenes y Guías de Ejercicios

Arcodaci, Tiziano (64637)
tarcodaci@itba.edu.ar

Boyaciyan, Felipe Ignacio (64026)
fboyaciyan@itba.edu.ar

Bridoux, Juan Ignacio (64092)
jbridoux@itba.edu.ar

Profesor
Agustín Golmar

Instituto Tecnológico de Buenos Aires (ITBA)
Autómatas, Teoría de Lenguajes y Compiladores (72.39)

Fecha de entrega: 14/06/2026

Índice

1	Introducción	1
2	Modelo Computacional	1
2.1	Dominio	1
2.2	Lenguaje	2
2.2.1	Características principales	2
2.2.2	Referencia de bloques y propiedades	2
2.2.3	Ejemplo representativo	4
2.2.4	Cambios respecto de la especificación del Stage I	6
3	Implementación	7
3.1	Frontend	7
3.1.1	Análisis Léxico (Flex)	7
3.1.2	Análisis Sintáctico (Bison)	7
3.1.3	Árbol de Sintaxis Abstracta (AST)	7
3.2	Backend	8
3.2.1	Análisis Semántico	8
3.2.2	Generación de Código	8
3.3	Dificultades Encontradas	9
4	Futuras Extensiones	9
5	Conclusiones	10

1. Introducción

Este trabajo consiste en el desarrollo de un lenguaje llamado **xam**, el cual permite crear exámenes y guías de ejercicios de forma sencilla y estandarizada haciendo uso de bloques simples y legibles. El compilador toma un archivo `.xam` y produce documentos $\text{\LaTeX}$ listos para compilar a PDF.

El proyecto fue desarrollado en el marco de la materia *Autómatas, Teoría de Lenguajes y Compiladores* (72.39) del ITBA, durante el primer cuatrimestre de 2026. La implementación se realizó en C, usando Flex y Bison [3], sobre la base del proyecto *Flex-Bison-Compiler* [1] (branch `production`, tag `v2.0.0`).

El objetivo es abstraer la complejidad del formateo manual en $\text{\LaTeX}$ al crear evaluaciones, permitiendo a docentes definir estos documentos de forma declarativa. El compilador se encarga de generar la estructura, el formato de los ejercicios, grillas de corrección y hojas de respuestas de manera automática.

2. Modelo Computacional

2.1. Dominio

El compilador tiene el objetivo de permitir a docentes desarrollar exámenes y guías de ejercicios de forma clara y sencilla a partir de una descripción declarativa, obteniendo como resultado documentos $\text{\LaTeX}$.

Para maximizar los métodos de evaluación disponibles, se incluyen diversos tipos de ejercicios (como Multiple Choice o True or False). De igual forma se ofrece la posibilidad de incluir elementos frecuentes en este tipo de documentos, como encabezados, imágenes, sistema de puntajes o incisos. Finalmente, es posible incluir una “Hoja de Respuestas”, así como también exportar diferentes exámenes a partir de un mismo código, facilitando la creación de “Temas” (dado que la aleatorización se resuelve en tiempo de compilación, ejecutar el compilador múltiples veces produce variantes distintas).

xam modela este dominio mediante las siguientes abstracciones:

- **Programa:** unidad de compilación que representa un examen o guía completa.
- **Header:** metadatos del documento (título, materia, profesor, fecha, duración, etc.).
- **Ejercicios:** unidades evaluativas tipadas (`multiplechoice`, `trueorfalse`, `openquestion`, `fillblanks`, `choosefrom`, `match`, `fillchart`).
- **Set:** agrupación de ejercicios como incisos de un mismo punto, con posibilidad de aleatorización.
- **Section:** agrupación de ítems con selección aleatoria de un subconjunto.
- **Items auxiliares:** texto libre (`text`), imágenes (`image`) y tablas (`chart`) que acompañan a los ejercicios.

El compilador produce un archivo `.tex` usando la clase `exam` [2]. Opcionalmente, si se declara `answer_sheet: true`, se genera un segundo archivo `-answers.tex` con la hoja de respuestas como documento separado.

2.2. Lenguaje

xam es un DSL declarativo con sintaxis basada en bloques delimitados por llaves, con propiedades separadas por punto y coma. Tiene un aspecto similar a un archivo de configuración estructurado pero con semántica orientada al dominio educativo. Cada ejercicio se expresa como un bloque con su tipo y sus propiedades, lo que hace que un archivo `.xam` sea fácil de leer y escribir por un docente sin conocimientos de programación.

2.2.1. Características principales

- **Declarativo:** el usuario describe *qué* quiere en el examen, no *cómo* formatearlo.
- **Tipado por ejercicio:** cada bloque posee propiedades específicas validadas semánticamente.
- **Composicional:** los ejercicios se pueden agrupar en `set` (incisos) y `section` (secciones con selección aleatoria).
- **Header opcional:** se puede crear un programa que sea solo ejercicios, sin encabezado.
- **Comentarios:** de línea (`//`) y de bloque (`/* ... */`).
- **Aleatorización:** tanto incisos (`shuffle`) como selección de ejercicios por sección (`select`).

2.2.2. Referencia de bloques y propiedades

A continuación se presenta una tabla de referencia con todos los bloques disponibles en xam, sus parámetros, tipos y reglas de validación. Cada ejercicio se expresa como un bloque con su tipo y sus propiedades, lo que hace que un archivo `.xam` sea fácil de leer y escribir por un docente sin conocimientos de programación.

Bloque	Parámetro	Tipo	Reglas
header	<code>title</code>	String	Obligatorio. Único.
	<code>subject</code>	String	Único.
	<code>professor</code>	String	Único.
	<code>date</code>	String	Único.
	<code>duration</code>	Integer	Único.
	<code>score_grid</code>	Boolean	Único. Si <code>true</code> , todos los ejercicios deben declarar <code>score</code> y la suma debe ser 10 o 100.

Continúa en la siguiente página...

Bloque	Parámetro	Tipo	Reglas
	answer_sheet	Boolean	Único. Si true , todos los ejercicios deben declarar su respuesta correcta.
	student_name	Boolean	Único.
	student_id	Boolean	Único.
	course	Boolean	Único.
	instructions	String	Repetible.
multiplechoice	question	String	Único. Obligatorio.
	option	String/Int	Mínimo 2. # marca la correcta.
	score	Integer	Único.
trueorfalse	question	String	Único. Obligatorio.
	justify	Boolean	Único.
	answer	Boolean	Único.
	score	Integer	Único.
openquestion	question	String	Único. Obligatorio.
	lines	Integer	Único. No negativo.
	answer	String	Único.
	score	Integer	Único.
fillblanks	question	String	Único. Obligatorio. Debe contener al menos un _.
	answer	String	Repetible.
	score	Integer	Único.
match	question	String	Único.
	pair	[String, String]	Mínimo 2 pares.
	score	Integer	Único.
choosefrom	task	String	Único.
	options	[String, ...]	Único. Mínimo 2. Más opciones que blanks.
	text	String	Único. Blanks con _.
	answer	[Integer, ...]	Único.
	shuffle	Boolean	Único.
	score	Integer	Único.
fillchart	task	String	Único.
	dim	[Int, Int]	Único. Filas y columnas.
	(r,c)	String	Contenido pre-llenado de celda.

Continúa en la siguiente página...

Bloque	Parámetro	Tipo	Reglas
	answer	[String, ...]	Único.
	score	Integer	Único.
set	text	String	Único. Enunciado del ejercicio.
	shuffle	Boolean	Único. Aleatoriza incisos.
	score	Integer	Único. Scores internos se ignoran.
	<i>bloques</i>	Block	Cualquier ejercicio anidado.
section	select	Integer	Cantidad de ítems a seleccionar aleatoriamente.
	<i>ítems</i>	Item	Cualquier ítem anidado.
image	path	String	Único. Ruta de la imagen.
	caption	String	Único. Epígrafe.
	width	Integer	Único. Ancho en %.
chart	task	String	Único. Texto acompañante.
	dim	[Int, Int]	Único.
	(r,c)	String	Contenido de celda.

El bloque **chart** es un ítem visual (tabla que acompaña al ejercicio), mientras que **fillchart** es un ejercicio donde el alumno debe completar la tabla.

2.2.3. Ejemplo representativo

El siguiente programa define un parcial de la materia con grilla de puntuación, múltiples ejercicios de desarrollo y una tabla auxiliar con la función de transición de un autómata:

```
header {
  title: "Parcial 1";
  subject: "Automatas, Teoria de Lenguajes y Compiladores
    (72.39)";
  professor: "Lic. Ana Maria Arias Roig";
  score_grid: true;
  student_name: true;
  student_id: true;
  instructions: "Condicion minima de aprobacion: Acumular 6
    puntos.";
  instructions: "Todos los ejercicios deben resolverse
    utilizando los algoritmos vistos en clase.";
  instructions: "El examen se evalua por lo que esta escrito.";
}
```

```

openquestion {
  score: 2;
  question: "El conjunto  $P \subseteq \Sigma^*$ , donde  $\Sigma = \{a, b\}$  se define recursivamente de la siguiente manera :  $\begin{itemize} \item \lambda \in P \item \text{si } x \in P \rightarrow axb \in P \end{itemize}$  Demostrar por induccion estructural que  $P \subseteq L$  siendo  $L = \{ \omega \in \{a, b\}^* / \#_a(\omega) = \#_b(\omega) * 2 \wedge \omega = \omega^R \wedge \#_b(\omega) \geq 0 \}$ .";
  lines: 0;
}

openquestion {
  score: 3;
  question: "Mostrar que la ER  $(abb^*a)^*a(ab)^*$  equivale a L (M) siendo:  $M = \langle \{a, b\}, \{p, q, r, s, t, u\}, \delta, p, \{u\} \rangle$ , con  $\delta$  dada por la siguiente tabla:";
  lines: 0;
}

chart {
  task: "";
  dim: [7, 4];
  (1,1): " $\delta$ ";
  (1,2): " $a$ ";
  (1,3): " $b$ ";
  (1,4): " $\lambda$ ";
  (2,1): " $\rightarrow p$ ";
  (2,2): " $\{q\}$ ";
  (2,3): " $\emptyset$ ";
  (2,4): " $\{s\}$ ";
  (3,1): " $q$ ";
  (3,2): " $\emptyset$ ";
  (3,3): " $\{r\}$ ";
  (4,1): " $r$ ";
  (4,2): " $\{p\}$ ";
  (4,3): " $\{r\}$ ";
  (5,1): " $s$ ";
  (5,2): " $\{q, u\}$ ";
  (5,3): " $\emptyset$ ";
  (6,1): " $t$ ";
  (6,2): " $\emptyset$ ";
  (6,3): " $\{u\}$ ";
  (7,1): " $*u$ ";
  (7,2): " $\{t\}$ ";
  (7,3): " $\emptyset$ ";
}

openquestion {
  score: 3;

```

```

question: "Construir un AFD Minimo $M$ que acepte $L = \{\omega \in \{a,b,c\}^* / \omega = a^nb^mc^p$ con $n+m \equiv 0(2) \wedge p \equiv 0(3)\}$.";
lines: 0;
}

```

Código 1: Parcial completo en xam.

2.2.4. Cambios respecto de la especificación del Stage I

Durante la implementación, la sintaxis del lenguaje evolucionó respecto de la propuesta inicial. Los cambios principales fueron:

1. **Renombrado de keywords:** los nombres abreviados originales (`mc`, `torf`, `direct`, `blanks`, `choose_from`) fueron reemplazados por sus formas completas (`multiplechoice`, `trueorfalse`, `openquestion`, `fillblanks`, `choosefrom`). Esto mejora la legibilidad y reduce la ambigüedad.
2. **Separación entre `chart` y `fillchart`:** se introdujeron dos construcciones. `chart` es un ítem visual (una tabla que acompaña al ejercicio pero no es el ejercicio en sí), mientras que `fillchart` es un ejercicio donde el alumno debe completar la tabla. Esto permite, por ejemplo, incluir una tabla de transiciones como contexto de un `openquestion`.
3. **Propiedad `professor` en el header:** se agregó para permitir que el examen incluya el nombre del docente a cargo. Es un campo que se había omitido en la primera iteración.
4. **Propiedad `instructions repetible`:** en lugar de un único string, ahora `instructions` puede aparecer múltiples veces en el header. Cada ocurrencia se renderiza como un ítem separado en la lista de instrucciones del examen.
5. **Bloque `image`:** se agregó a sugerencia de la cátedra. Permite incluir imágenes con propiedades `path`, `caption` y `width`.
6. **Answer sheet como documento separado:** la hoja de respuestas se genera como un archivo `.tex` independiente (con sufijo `-answers`), en lugar de ser una sección dentro del mismo documento del examen.
7. **Eliminación de secuencias de escape propias para símbolos:** en la especificación original se habían propuesto secuencias como `\->` para flechas, `\aleph0` para $\aleph_0$, etc. Durante la implementación se optó por no implementar un sistema de escape propio y en su lugar permitir que el docente escriba directamente código $\text{\LaTeX}$ entre signos `$...$` dentro de los strings. Si bien esto requiere que el usuario conozca la sintaxis de $\text{\LaTeX}$ para expresiones matemáticas, simplificó considerablemente la implementación del compilador al evitar una fase adicional de traducción de símbolos.

3. Implementación

El desarrollo se dividió en tres etapas: diseño del lenguaje, frontend y backend.

3.1. Frontend

El frontend está implementado en C usando Flex para el análisis léxico y Bison para el análisis sintáctico.

3.1.1. Análisis Léxico (Flex)

El analizador léxico reconoce tres categorías principales de lexemas:

- **Palabras clave:** los tipos de ejercicios (`multiplechoice`, `trueorfalse`, `openquestion`, `fillblanks`, `choosefrom`, `match`, `chart`, `fillchart`, `set`, `section`, `image`) y las propiedades (`question`, `answer`, `score`, `option`, `pair`, `dim`, etc.).
- **Literales:** enteros, strings delimitados por comillas dobles y booleanos (`true/false`).
- **Delimitadores:** `{ }` `[ ]` `( )` `:` `;` `,` `#`.

Se implementaron dos contextos de Flex: `STRING_LITERAL` para manejar correctamente las cadenas (contenido arbitrario entre comillas) y `MULTILINE_COMMENT` para los comentarios de bloque, junto con soporte para comentarios de línea con `//`.

3.1.2. Análisis Sintáctico (Bison)

La gramática define un programa como un bloque `header` opcional seguido de una secuencia de ítems. Las decisiones de diseño principales fueron:

- **Propiedades en cualquier orden:** cada bloque se parsea como una secuencia de propiedades clave-valor que se acumulan incrementalmente en el nodo AST. El docente puede escribirlas en el orden que prefiera.
- **Recursión a izquierda:** las listas de propiedades, ítems y ejercicios dentro de sets usan recursión a izquierda para evitar desbordamientos de pila en documentos grandes.
- **Items heterogéneos:** ejercicios, secciones, texto, imágenes y tablas `chart` están unificados bajo una misma estructura `Item` con un discriminador de tipo.

3.1.3. Árbol de Sintaxis Abstracta (AST)

El AST se define como un conjunto de structs C enlazados. El nodo raíz `Program` contiene un puntero opcional a `Header` y una lista enlazada de `Item`. Cada `Item` puede ser un ejercicio (con un discriminated union sobre los ocho tipos soportados), una sección, un texto, una imagen o un `chart`. Esta representación permite que el backend opere de forma uniforme sin conocer los detalles sintácticos.

Toda la memoria del AST se libera recursivamente al finalizar la compilación.

3.2. Backend

El backend recibe el AST producido por el frontend y opera en dos etapas: análisis semántico y generación de código.

3.2.1. Análisis Semántico

El analizador semántico aplica validaciones específicas del dominio que no pueden expresarse en la gramática. Las reglas implementadas cubren los siguientes aspectos:

- **Propiedades obligatorias:** si el examen declara un header, el campo `title` es obligatorio. Cada tipo de ejercicio exige al menos una `question` (o `task` según el tipo).
- **Cardinalidades mínimas:** `multiplechoice` requiere al menos dos opciones; `match` requiere al menos dos pares; `choosefrom` requiere al menos dos opciones disponibles.
- **Coherencia de blanks:** el campo `question` o `text` debe contener al menos un grupo de `_` que representen los blancos a completar. En `choosefrom`, además, la cantidad de opciones debe ser mayor o igual a la cantidad de blancos.
- **Coherencia de respuestas:** en `choosefrom`, si se declara `answer`, la cantidad de respuestas debe coincidir con la cantidad de blancos en `text`.
- **Celdas dentro de dimensiones:** en `fillchart`, cada celda declarada con coordenadas `(r,c)` debe estar dentro de las dimensiones indicadas por `dim`.
- **Grilla de puntajes:** cuando `score_grid: true`, todos los ejercicios deben declarar `score` y la suma total debe ser exactamente 10 o 100.
- **Hoja de respuestas:** cuando `answer_sheet: true`, cada ejercicio que lo permita debe declarar su respuesta correcta; en `multiplechoice`, al menos una opción debe estar marcada con `#`.
- **Propiedades duplicadas:** se detecta la declaración duplicada de propiedades como `score` o `question` dentro de un mismo bloque.

El analizador acumula todos los errores encontrados en una lista enlazada de `SemanticError` en lugar de detenerse al primero, de modo que el compilador puede reportar todos los problemas de un archivo en una sola ejecución.

3.2.2. Generación de Código

El generador de código produce archivos `LATEX` a partir del AST validado. Se utiliza la clase `exam` [2], que provee ambientes nativos para preguntas, opciones, incisos y grillas de puntuación. Las decisiones de diseño principales:

- **Salida a archivo:** el generador escribe directamente a un `FILE *` con un sistema de indentación con tabs para producir código legible, pensando también en que el docente tenga un código `LATEX` que pueda fácilmente editar.
- **Archivo de respuestas separado:** cuando se habilita `answer_sheet`, se genera un segundo documento `LATEX` con sufijo `-answers.tex`.
- **Aleatorización:** tanto el shuffle de incisos (`set`) como la selección aleatoria de ítems (`section` con `select`) se resuelven en tiempo de compilación usando Fisher-Yates shuffle.
- **Match con column desordenada:** en matching, la columna derecha se mezcla automáticamente para que el alumno deba realizar la correspondencia.

Para cada tipo de ejercicio existe una función dedicada de generación.

3.3. Dificultades Encontradas

1. **Strings en Flex:** inicialmente los strings no soportaban contenido con caracteres especiales. Se resolvió usando un start condition exclusivo que captura todo entre comillas.
2. **Propiedades en orden arbitrario:** la primera versión de la gramática exigía un orden fijo para las propiedades. Se refactorizó a reglas donde cada propiedad se acumula incrementalmente, permitiendo cualquier orden.
3. **Detección de propiedades duplicadas:** dado que las propiedades se aceptan en cualquier orden, el parser no puede rechazar duplicados por gramática. La solución fue detectarlos en las acciones semánticas de Bison: antes de asignar un valor, se verifica si el campo ya fue seteado y, de ser así, se marca un flag global de error. Esto permite reportar el duplicado sin abortar el parsing.
4. **Diferencia entre chart ítem y fillchart ejercicio:** durante la implementación se identificó la necesidad de distinguir una tabla que acompaña al ejercicio (contexto visual) de un ejercicio donde el alumno completa la tabla. Esto motivó la separación en dos bloques.
5. **Generación de la answer sheet:** mantener la correlación entre ejercicios y respuestas en un documento separado requirió recorrer el AST una segunda vez, filtrando solo ejercicios con respuesta declarada.
6. **Selección aleatoria en section:** implementar `select` correctamente implicó recolectar los ítems en un arreglo temporal, aplicar Fisher-Yates y generar solo los primeros N .

4. Futuras Extensiones

1. **Generación de variantes:** generar múltiples versiones del mismo examen con distintas semillas de aleatorización en una misma compilación, útil para crear “Temas”

automáticamente.

2. **Import de archivos:** permitir dividir un examen en múltiples archivos `.xam` e importarlos, facilitando la reutilización de bancos de preguntas.
3. **Expresiones matemáticas nativas:** actualmente los símbolos matemáticos se escriben como `LATEX` crudo dentro de strings. Una extensión sería un mini-lenguaje de expresiones que se traduzca automáticamente.
4. **Ponderación de dificultad:** agregar una propiedad `difficulty` y un algoritmo que seleccione ejercicios balanceando la dificultad total.
5. **Mensajes de error con línea y columna:** actualmente los errores semánticos no indican la posición en el fuente. Integrar las ubicaciones de Bison en los nodos del AST permitiría errores más informativos.
6. **Contenido estructurado en strings:** actualmente, para incluir listas o sub-ítems dentro del enunciado de un ejercicio, el docente debe escribir comandos `LATEX` directamente en el string. Una extensión sería soportar sintaxis nativa para listas y contenido enriquecido dentro de las propiedades de texto.

5. Conclusiones

El desarrollo de Xam consistió en la implementación de un compilador para un DSL orientado a la confección de exámenes académicos. El lenguaje permite al docente describir el contenido de un examen —tipos de ejercicio, propiedades, puntajes, respuestas correctas— sin intervención directa en `LATEX`, delegando al compilador la responsabilidad de producir un documento tipográficamente correcto. Cuando se lo indica mediante el encabezado, el compilador genera además una hoja de respuestas como documento `LATEX` independiente.

Una de las decisiones de diseño que más impactó en la calidad del DSL fue la elección de palabras clave completas sobre abreviaciones, y una sintaxis de bloques con propiedades explícitas. Esto hace que un archivo `.xam` resulte legible incluso sin conocer el lenguaje de antemano, lo cual es especialmente relevante para un DSL cuyo usuario objetivo no es un programador. Esta misma legibilidad facilitó la escritura de los casos de prueba y la detección de errores semánticos durante el desarrollo.

El analizador semántico es una pieza central del compilador. La naturaleza del dominio impone restricciones que van más allá de lo que puede expresarse en una gramática libre de contexto: puntajes que deben sumar exactamente 10 o 100, coherencia entre la cantidad de blancos y las opciones disponibles en `choosefrom`, o la consistencia de la hoja de respuestas con los ejercicios declarados. Implementar estas validaciones como una fase independiente, con acumulación de errores en lugar de fallo temprano, mantuvo el código organizado y mejoró la experiencia de uso del compilador.

Referencias

- [1] Agustín Golmar. *Flex-Bison-Compiler*. Proyecto base, ITBA. 2024. URL: <https://github.com/agustin-golmar/Flex-Bison-Compiler>.
- [2] Philip Hirschhorn. *Using the exam document class*. 2017. URL: <https://ctan.org/pkg/exam>.

Bibliografía

- [3] John Levine. *flex & bison*. O'Reilly Media, 2009.